

Logistic Regression with L1 penalty using Cyclical Coordinate Descent

Marek Mytkowski, Julia Dudzińska, Weronika Gozdera

31 March 2025

1 Motivation

The goal of the project was to implement Cyclic Coordinate Descent (CCD) algorithm for parameter estimation in regularized logistic regression with L1 (lasso) penalty and compare it with standard logistic regression model without regularization.

2 Dataset preparation

2.1 Selected datasets

To test the performance of the implemented algorithm, we have selected four datasets available on the OpenML platform. During dataset selection, we have focused on datasets with many features. The table below illustrates the main dataset characteristics: number of samples, number of features, features to samples ratio (before data preprocessing), and balance between classes (after data preprocessing – accounts for reduction from multiclass to binary classification in case of Arrhythmia dataset). Each of the selected datasets is briefly described below the table.

Dataset	Samples (n)	Features (d)	d/n ratio	Class Balance
Arcene	200	10,000	50.0	56%/44%
Arrhythmia	452	279	0.62	54%/46%
Bioresponse	3,751	1,776	0.47	46%/54%
Madelon	2,600	500	0.19	50%/50%

Table 1: Characteristics of the selected datasets

Arcene [1] is a dataset that consists of mass-spectrometric data for cancer detection. We have chosen this dataset as it is extremely high-dimensional, which should be a good test for the implemented regularization. During data preprocessing, 39 constant features and 6547 correlated features were removed.

Arrhythmia [2] dataset contains clinical features for classifying cardiac arrhythmia. The original multi-class problem of detecting the presence of cardiac arrhythmia and classifying it into one of 16 groups was converted to binary by merging minority classes. It was selected based on its features-to-sample ratio.

Bioresponse [3] describes molecular activity using their chemical properties (molecular descriptors) – binary prediction relates to whether a given molecule elicits a specific biological response. Similarly to Arrhythmia dataset, it was selected because of the d/n ratio.

Madelon [4] is an artificial dataset designed for feature selection challenges. It contains 500 features, among which only 20 are relevant (5 informative, 15 redundant). The goal of selecting this dataset was to test whether the method can correctly identify the 5 informative true features.

2.2 Preprocessing

All datasets were preprocessed following the same steps to make sure both project requirements and fundamental assumptions of logistic regression are satisfied. Categorical features were label encoded. Missing values were handled using mean imputation. We eliminated feature pairs with Pearson correlation $\rho > 0.90$ and zero-variance features. All numerical features were standardized to zero mean and unit variance ($z = \frac{x-\mu}{\sigma}$). For datasets where the feature dimensionality d did not meet the $d \geq 0.5n$ condition, we generated additional features through random permutations of existing ones. The target variable was binarized when necessary by converting the majority class to class 0 and the rest of the classes to class 1.

2.3 Synthetic dataset

The synthetic dataset was created by sampling from multivariate normal distributions with parameters (n, p, d, g) , where n controls sample size, p determines class balance, d specifies dimensionality, g – correlation. Class distributions have mean vectors $\mu_0 = \mathbf{0}$ and $\mu_1 = [1, 1/2, \dots, 1/d]^\top$, with shared covariance matrix $\Sigma_{ij} = g^{|i-j|}$.

3 Implementation

3.1 Theoretical considerations

In logistic regression, we optimize the mean of the maximum log-likelihood function. As in the reference article [5], we construct a quadratic approximation $l_Q(\beta_0, \beta)$ and minimize it using coordinate descent:

$$\min_{(\beta_0, \beta) \in \mathbb{R}^{p+1}} \{-l_Q + \lambda \|\beta\|_1\}. \quad (1)$$

In cyclical coordinate descent, each step minimizes along a single direction. We can efficiently find the optimal direction at each step by leveraging the quadratic approximation and applying the soft thresholding function to handle the non-differentiable LASSO penalty terms. We compute the gradient at $\beta_j = \tilde{\beta}_j$ and assume $\tilde{\beta}_j > 0$ to simplify the LASSO penalty:

$$\begin{aligned} \left. \frac{\partial(-l_Q + \lambda\|\beta\|_1)}{\partial\beta_j} \right|_{\beta=\tilde{\beta}} &= \left. \frac{\partial\left(\frac{1}{2N} \sum_{i=1}^N w_i(z_i - \beta_0 - x_i^T \beta)^2 + C(\tilde{\beta}_0, \tilde{\beta}) + \lambda\|\beta\|_1\right)}{\partial\beta_j} \right|_{\beta=\tilde{\beta}} \\ &= -\frac{1}{N} \sum_{i=1}^N w_i x_{ij} (z_i - \tilde{\beta}_0 - \sum_{z=1, z \neq j}^p x_{iz} \tilde{\beta}_z - x_{ij} \tilde{\beta}_j) + \lambda = 0. \end{aligned} \quad (2)$$

where $z_i = \tilde{\beta}_0 + x_i^T \tilde{\beta} + \frac{y_i - \tilde{p}(x_i)}{\tilde{p}(x_i)(1 - \tilde{p}(x_i))}$ and $w_i = \tilde{p}(x_i)(1 - \tilde{p}(x_i))$. After further simplifications, we obtain:

$$\tilde{\beta}_j = \frac{\sum_{i=1}^N w_i x_{ij} (z_i - \tilde{\beta}_0 - \sum_{z=1, z \neq j}^p x_{iz} \tilde{\beta}_z) - N\lambda}{\sum_{i=1}^N w_i x_{ij}^2}. \quad (3)$$

Using the soft thresholding function (separately considering cases where $\tilde{\beta}_j > 0$, $\tilde{\beta}_j < 0$, and $\tilde{\beta}_j = 0$), we obtain:

$$\tilde{\beta}_j = \frac{S\left(\sum_{i=1}^N w_i x_{ij} (z_i - \tilde{\beta}_0 - \sum_{z=1, z \neq j}^p x_{iz} \tilde{\beta}_z), N\lambda\right)}{\sum_{i=1}^N w_i x_{ij}^2}. \quad (4)$$

To efficiently implement this in Python, we vectorize Eq. (4). Notably, when inserting z_i into the first argument of the soft thresholding function, we derive:

$$\sum_{i=1}^N w_i x_{ij} (\tilde{\beta}_0 + x_i^T \tilde{\beta} + \frac{y_i - \tilde{p}(x_i)}{\tilde{p}(x_i)(1 - \tilde{p}(x_i))} - \tilde{\beta}_0 - x_i^T \tilde{\beta} + x_{ij} \tilde{\beta}_j) = \sum_{i=1}^N x_{ij} (y_i - \tilde{p}(x_i)) + \tilde{\beta}_j \sum_{i=1}^N w_i x_{ij}^2. \quad (5)$$

3.2 Implementation details

As the reference article recommends, we update l_Q and the current estimates of β once per full cycle. This approach enables the precomputation of the vector $y_i - \tilde{p}(x_i)$ and the summation $\sum_{i=1}^N w_i x_{ij}^2$ once per cycle, improving computational efficiency. The single-cycle implementation relies exclusively on matrix operations, avoiding additional loops, which optimizes cycle computations in Python. Furthermore, as recommended in the reference paper, we employ constant weight values set to 0.25 for optimisation purposes.

For the regularization parameter λ , we use a user-defined maximum value $\lambda_{\max}$ and a small positive threshold ε . We generate K values of λ spanning the log-scale range $[\varepsilon\lambda_{\max}, \lambda_{\max}]$. If $\tilde{p}(x_i)$ is within ε_1 of 0 or 1, we round it and set $w_i = \varepsilon_1$. We use coefficients obtained using bigger lambda to warm start the next step's calculations for smaller lambda. We use a zero vector as the initial coefficients guess for $\lambda_{\max}$.

The stopping criterion for iterations is:

$$|(-l_Q^{(j+1)} + \lambda\|\beta^{j+1}\|_1) - (-l_Q^{(j)} + \lambda\|\beta^j\|_1)| < \varepsilon_2, \quad (6)$$

where $\varepsilon_2 = 10^{-10}$ by default. If it is not satisfied during $I_{\max}$ (by default 10000) iterations, calculations are stopped for the current lambda.

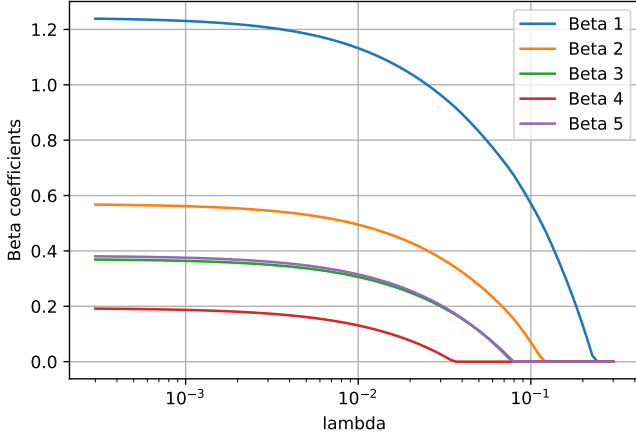
3.3 Implementation correctness

To validate the correctness of our algorithm's implementation, we plotted the regularization path, which represents the coefficient values as a function of λ on a logarithmic scale. We then compared this with the regularization path obtained using scikit-learn's `LogisticRegression` class, evaluated for the same set of λ values and plotted in an identical manner. A slight discrepancy was observed, likely due to scikit-learn employing a different loss function formulation—specifically, using the sum rather than the mean of the maximum log-likelihood function. This difference affects the scaling of λ ; however, identical results can be achieved if $C = \frac{1}{\lambda n}$, where λ corresponds to the value produced by our algorithm. Figure 1 illustrates the resulting regularisation paths.

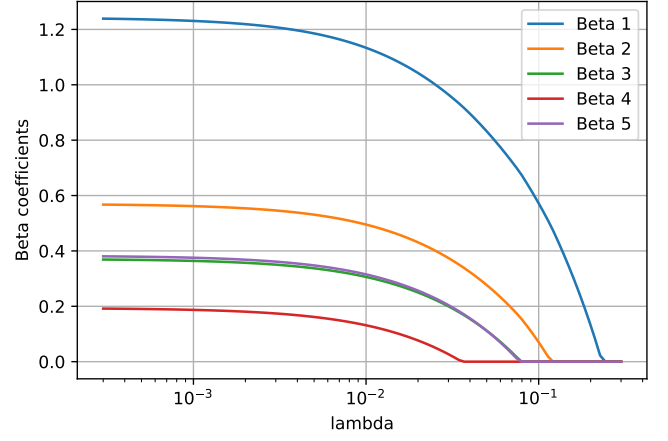
We also evaluated the performance of our implementation in the absence of regularization ($\lambda = 0$) and compared it to scikit-learn's `LogisticRegression` with `penalty=None`. The classification accuracy on the test dataset was identical in both cases, approximately 0.74. The corresponding coefficient values match with a tolerance of 10^{-3} .

To examine the changes in the loss function and coefficient values across iterations, we plotted these metrics; they are presented in Figure 2. The loss function, defined as the negative mean of the maximum log-likelihood function, decreases until the previous quadratic approximation is within a tolerance of ε_2 of the current approximation. The β coefficients stabilize after a few iterations, which is the expected result.

Based on the results of these three experiments, we conclude that our implementation is correct.



(a) Regularization path obtained using scikit-learn's `LogisticRegression`.



(b) Regularization path obtained using our cyclic coordinate descent implementation.

Figure 1: Comparison of regularization paths on the synthetic dataset with parameters $p = 0.5$, $n = 1000$, $d = 5$, and $g = 0$. The `LogisticRegression` model is configured with `penalty='l1'`, `solver='liblinear'`, and `max_iter=10000` to match the settings of our implementation. The λ values are generated within the fitting process of our implementation and subsequently passed to `LogisticRegression` as $C = \frac{1}{\lambda n}$. The results are then plotted as a function of λ rather than C . The same colour on both plots indicates corresponding coefficients (of the same features).

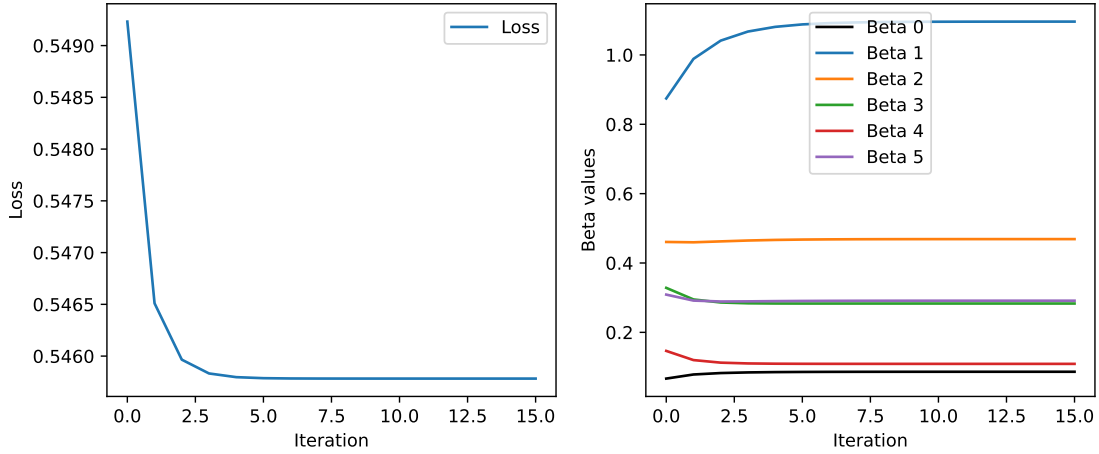


Figure 2: Loss (minus the mean of maximum log-likelihood for the training split) and coefficient values over iteration for best lambda ($\lambda \approx 0.0184$), without warm start, on the synthetic dataset with $p = 0.5$, $n = 1000$, $d = 5$.

4 Experiments

4.1 Comparison on synthetic dataset

For the performance comparison of the implemented method with the standard implementation of logistic regression without regularization on synthetic data, the `LogisticRegression` method with `penalty='None'` and default values of the remaining parameters from the *sklearn* library was used.

The algorithms were evaluated against the following metrics: AUC ROC and balanced accuracy. In order to fully understand the capabilities of the implemented method, in addition to the results of the default implementation of the `fit` method, the following section also includes values obtained after selecting the optimal coefficients with respect to the tested measure (the optimization of the appropriate measure was performed on the validation set).

The data sets were generated according to the method described in section 2.3 and split in proportions 50-20-30 for training-validation-test sets. The basic generator settings were established – $n = 1000$, $p = 0.5$, $d = 50$, $g = 0$. Such settings ensure obtaining a balanced data set of non-correlating features of average size and with a reasonable ratio of the number of observations to the number of features. In order to determine the influence of individual variables on the performance of the tested methods, four independent analyses were conducted, in which each variable was changed within a set range while leaving the values of the remaining variables constant, equal to the basic parameters. In each analysis, experiments were conducted for twenty equally spaced values of a given variable in the tested range. In this way, the values of the given metrics were compared for $n \in [100, 10000]$, $p \in [0.1, 0.9]$, $d \in [10, 500]$ and $g \in [0, 0.9]$. The obtained results are presented in Figures 3 and 4.

In general, logistic regression performance increases with the number of observations. It is noteworthy that

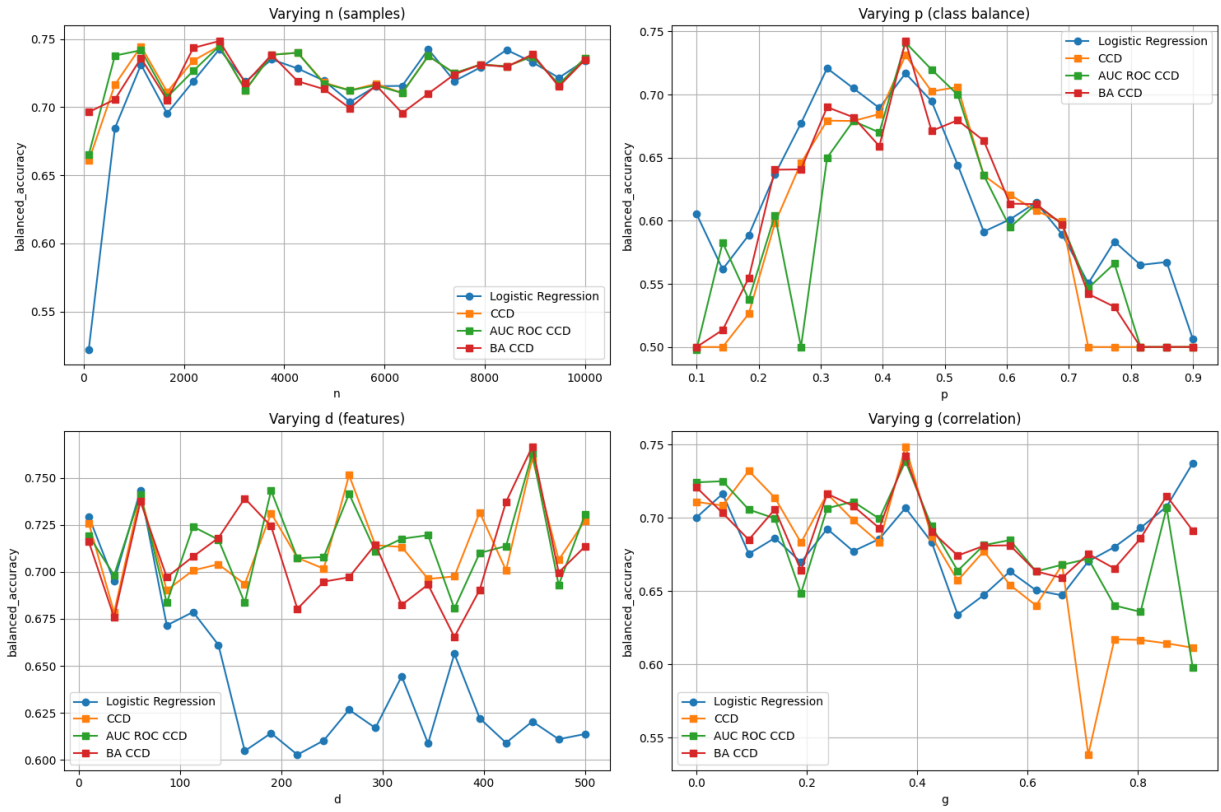


Figure 3: The influence of n , p , d , g (synthetic data generator parameters) on balanced accuracy. The blue line shows the results for `LogisticRegression` from *sklearn*, the orange – `LogRegCCD`, the green – `LogRegCCD` after choosing the λ that minimizes AUC ROC, the red – `LogRegCCD` after choosing the λ that minimizes balanced accuracy.

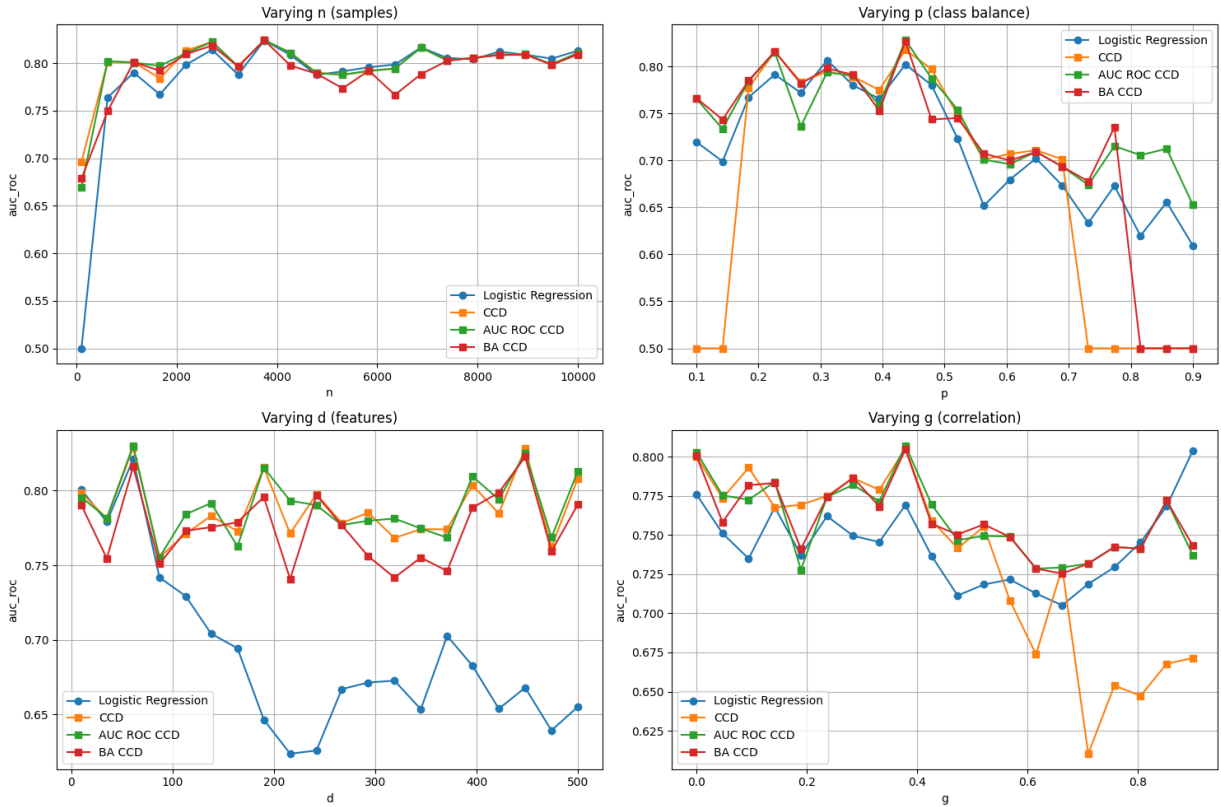


Figure 4: The influence of n , p , d , g (synthetic data generator parameters) on AUC ROC. The blue line shows the results for `LogisticRegression` from *sklearn*, the orange – `LogRegCCD`, the green – `LogRegCCD` after choosing the λ that minimizes AUC ROC, the red – `LogRegCCD` after choosing the λ that minimizes balanced accuracy.

LogisticRegression performed significantly worse at small n . Still, this difference became negligible around 3000 samples, where the methods stabilized, giving satisfactory results around 0.8 for ROC AUC and ranging between 0.7 and 0.75 for balanced accuracy.

Regression is sensitive to class imbalance. Performance peaks around $p = 0.5$ and drops for edge p values (less than 0.2 or higher than 0.7). For extreme imbalances, logistic regression without regularization often outperforms **LogRegCCD**, but for ROC AUC, choosing coefficients focused on minimizing the tested metric allows **LogRegCCD** to outperform **LogisticRegression**.

As the parameter d increases, the advantages of **LogRegCCD** stand out the most. The experiments showed that the method correctly eliminates features of low importance (in accordance with the data generation method – coefficients for features with the smallest indices usually had the highest values, further ones were more often zeroed). For the number of features greater than 100, a sharp decrease in the performance of **LogisticRegression** is visible, while **LogRegCCD** remains stable.

Increasing the value of the parameter g has a nonobvious effect on the performance of the methods. Initially, **LogRegCCD** gives better results, regardless of the choice of coefficients. However, around $g = 0.6$, the method’s performance decreases, while the performance of **LogisticRegression** increases. Choosing the optimal coefficients for the tested measure significantly increases the performance of **LogRegCCD** in this range of values, giving results competitive with **LogisticRegression**.

4.2 Comparison on real datasets

Data was preprocessed for the experiments on real datasets according to the process described in 2.2 and split in proportions: 70-10-20 for training-validation-test sets. Experiments were run for parameters $K = 20$, $\lambda_{max} = 0.5$, $I_{max} = 1000$ of the **LogRegCCD** function described in 3. The implemented algorithm was compared with **LogisticRegression** function from *sklearn* library, which was run with parameters *no penalty* and a maximum number of iterations equal to 1000 (same as I_{max}). Final coefficients from **LogRegCCD** function were obtained by validating against the ROC AUC measure.

Metric	LogRegCCD	LogisticRegression	Metric	LogRegCCD	LogisticRegression
ROC AUC	0.812	0.841	ROC AUC	0.681	0.666
PR AUC	0.875	0.881	PR AUC	0.594	0.570
F1 Score	0.824	0.649	F1 Score	0.693	0.630
Balanced Accuracy	0.750	0.719	Balanced Accuracy	0.673	0.627

(a) Arcene Dataset

Metric	LogRegCCD	LogisticRegression	Metric	LogRegCCD	LogisticRegression
ROC AUC	0.793	0.714	ROC AUC	0.657	0.540
PR AUC	0.810	0.736	PR AUC	0.645	0.531
F1 Score	0.793	0.735	F1 Score	0.632	0.505
Balanced Accuracy	0.749	0.693	Balanced Accuracy	0.629	0.522

(c) Bioresponse Dataset

(b) Arrhythmia Dataset

(d) Madelon Dataset

Figure 5: Comparison of **LogRegCCD** and **LogisticRegression** across the real datasets

For the Arcene dataset, the accuracy obtained on the test set was 0.775 for **LogRegCCD** and 0.675 for **LogisticRegression**. **LogRegCCD** performed substantial feature selection, which can be seen on Figure 6. The comparison of obtained measures is shown in Table 5a. Even when validating against the ROC AUC measure, **LogisticRegression** obtained results better by around 3 percentage points. A big difference can be seen for the F1 Score – **LogRegCCD** wins by around 18 percentage points. Considering a slight class imbalance shown in 1 and better result of **LogRegCCD** in the Balanced Accuracy measure (pointing at better handling of the imbalance and mitigating bias towards one class), these results suggest that **LogRegCCD** sacrificed some of the overall ranking quality for better generalization and class separation. Especially in the medical context, the performance of **LogRegCCD** would be considered superior. For the Arrhythmia dataset, Table 5b shows that **LogRegCCD** outperformed **LogisticRegression** against every measure, as well as in case of accuracy on the test set (**LogRegCCD** achieved 0.67 whereas **LogisticRegression** – 0.626). Regularization reduced 34 out of 244 features, so the percentage reduction was a lot less than compared to Arcene (as mentioned in the description of Figure 6).

Similarly, for the Bioresponse dataset, Table 5c shows even bigger improvements when applying regularization. For this dataset **LogRegCCD** achieved 0.758 accuracy on the test set, compared to 0.698 for **LogisticRegression**. Regularization reduced 818 out of 1875 features, a higher percentage than for Arrhythmia. This combination of performance and reduction suggests that the model was able to filter out noisy molecular descriptors and focus on the meaningful features influencing the biological response.

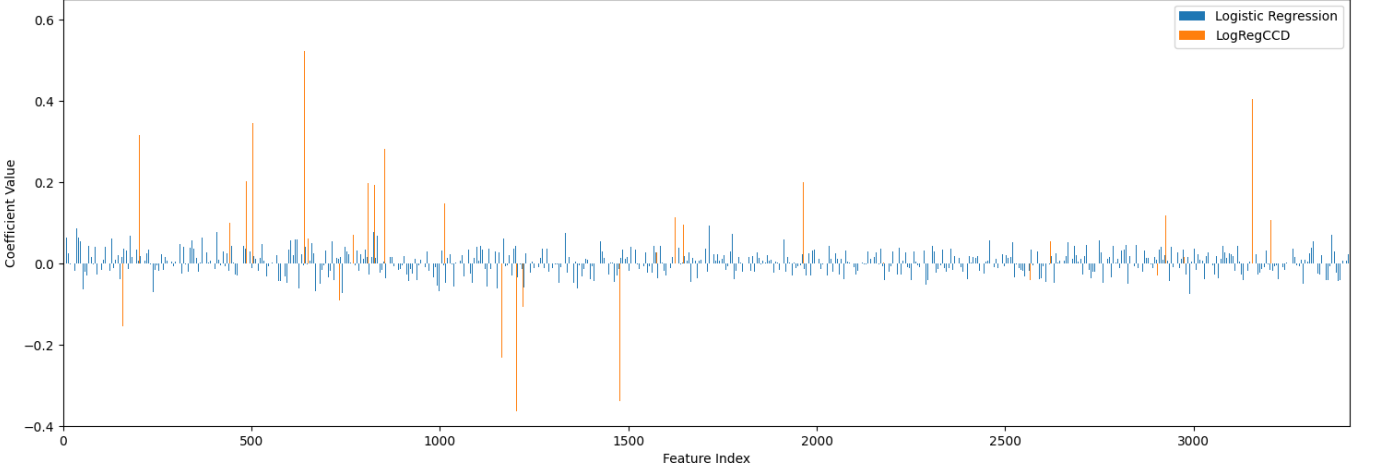
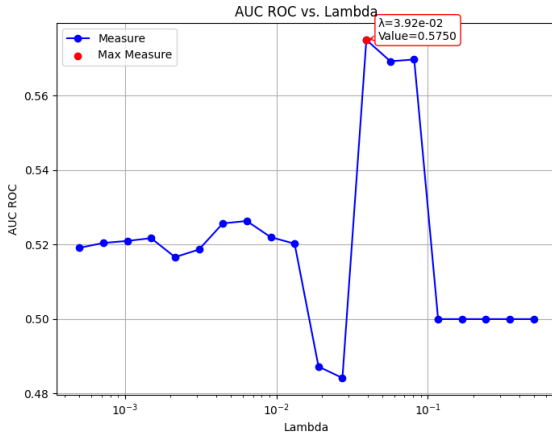
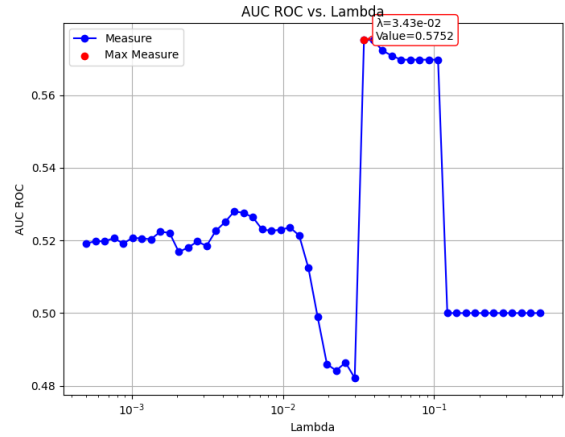


Figure 6: Regularization effects visualization for Arcene dataset – 3259 out of 3414 features were reduced. **LogRegCCD** coefficients correspond to $\lambda = 7.19 \cdot 10^{-4}$ which was chosen as the best value in the validation process.

LogisticRegression's performance on the Madelon dataset shown in Table 5d can be considered very weak (ROC AUC close to 0.5 suggests the classification is only slightly better than random guessing) – this model achieved 1.0 accuracy on the training set and 0.52 accuracy on the test set, suggesting strong overfitting. Regularization applied in **LogRegCCD** resulted in big improvements against all metrics (over 10 percentage points in each) and in test accuracy, achieving 0.629. As for reduced features, after validation of **LogRegCCD** there were 5 non-zero coefficients, 1 being the intercept. It is close to the expected result of keeping 5 informative coefficients. One of the 4 non-reduced features was a dummy feature – as 810 dummy features were added to comply with the $d \geq 0.5n$ condition and dummy generation was based on permuting existing features, there is a slight chance for some of them to correlate with existing ones or retain their informativity. We checked how many features would be selected if we increased K to 50, and a smaller λ was chosen during validation, increasing the number of meaningful features to 7. This and the plots of ROC AUC measure in Figure 7 during validation suggest high sensitivity to small changes in λ in the region from which the selected λ comes.



(a) ROC AUC measure plot for $K = 20$ lambdas



(b) ROC AUC measure plot for $K = 50$ lambdas

Figure 7: Comparison of validation results for Madelon dataset for different values of K . The sharp slope between λ corresponding to max measure and the first point on the left of it suggests big sensitivity to λ change in that region.

References

- [1] Guyon et al. (2008). *Arcene Dataset*. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/ml/datasets/Arcene>
- [2] Güler et al. (2005). *Arrhythmia Dataset*. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/ml/datasets/Arrhythmia>
- [3] Molecular AI (2012). *Bioresponse Dataset*. Kaggle.
<https://www.kaggle.com/c/bioresponse>
- [4] Guyon et al. (2004). *Madelon Dataset*. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/ml/datasets/Madelon>
- [5] Friedman, J. H., Hastie, T., Tibshirani, R. (2010). *Regularization Paths for Generalized Linear Models via Coordinate Descent*. Journal of Statistical Software, **33**(1), 1–22.
<https://doi.org/10.18637/jss.v033.i01>